

# P1. ESP32-S3 + MPU6050

## Adquisición y visualización de actividad humana



## Propósito de la práctica

Integrar progresivamente el sistema TinyWear. Primero se verificará la comunicación I2C y el funcionamiento del MPU6050; después, se adquirirán y visualizarán las seis señales de la IMU en Arduino Serial Plotter; finalmente, se incorporará la comunicación Wi-Fi directa mediante UDP unicast para transferir las muestras a una interfaz en Python y generar archivos CSV.

## Resultados de aprendizaje

- Conectar el MPU6050 al ESP32-S3 y verificar el bus I<sup>2</sup>C mediante escaneo y lectura de WHO\_AM\_I.
- Interpretar las seis señales de la IMU: ax, ay, az, gx, gy y gz.
- Adquirir y visualizar las seis señales a 50 Hz utilizando Arduino Serial Plotter.
- Relacionar los patrones de aceleración y de velocidad angular con: reposo/de pie, caminar y sentarse/levantarse.
- Establecer una comunicación Wi-Fi directa entre TinyWear y la laptop utilizando UDP unicast.
- Capturar muestras etiquetadas y guardarlas en archivos CSV con una longitud controlada.

## Materiales y preparación

- ✓ ESP32-S3 por equipo.
- ✓ Módulo MPU6050 por equipo.
- ✓ Cable USB de datos y jumpers Dupont.
- ✓ Clip, cuerda o soporte para fijar TinyWear a la cintura.
- ✓ Power bank o fuente de alimentación USB portátil.
- ✓ Computadora con [Arduino IDE](#) y [Python 3](#).
- ✓ Espacio para caminar algunos metros y una silla estable.



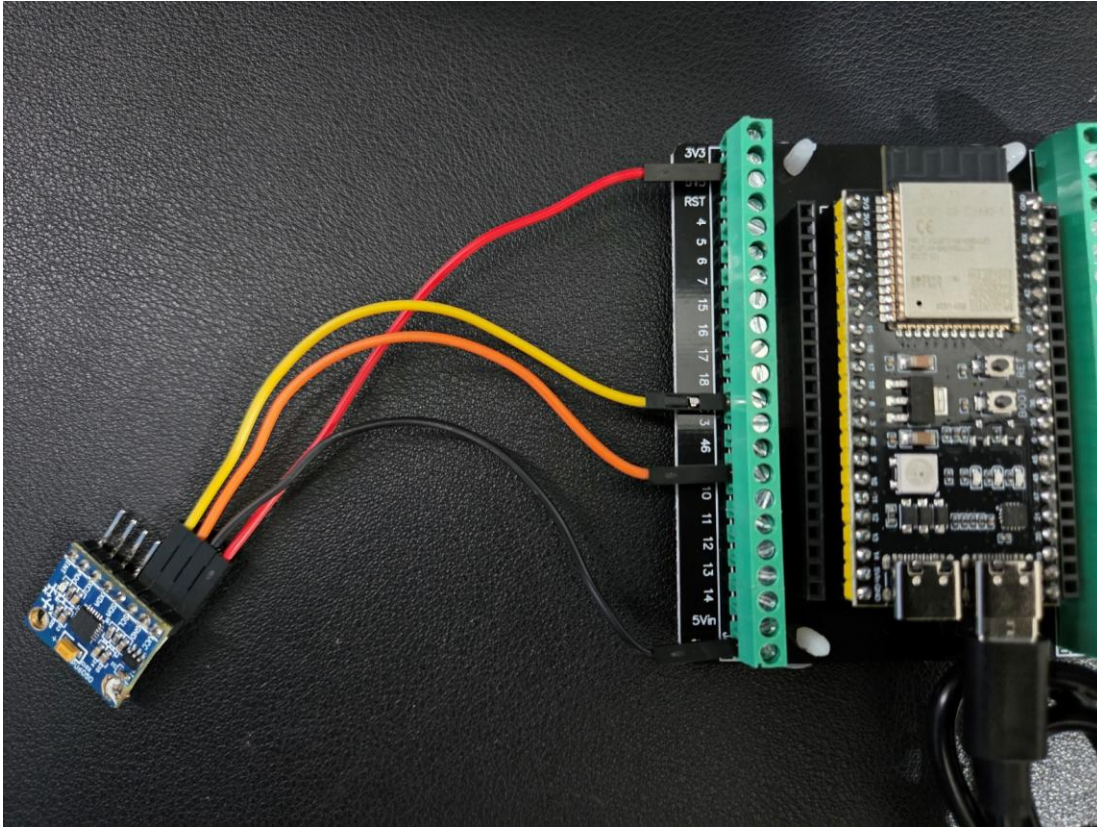
**Regla de trabajo:** no se requiere correr ni saltar. El wearable debe permanecer firme, sujeto y con la misma orientación durante las capturas.

## Conexión del MPU6050

La interfaz I2C utiliza los GPIO 8 (SDA) y 9 (SCL). Si la tarjeta ESP32-S3 utilizada tiene otra asignación, modifique SDA\_PIN y SCL\_PIN en el firmware.

| MPU6050 | ESP32-S3 | Función          |
|---------|----------|------------------|
| VCC     | 3.3 V    | Alimentación     |
| GND     | GND      | Referencia común |
| SDA     | GPIO 8   | Datos I2C        |
| SCL     | GPIO 9   | Reloj I2C        |

## Fotografía de referencia



## Parámetros comunes de adquisición

| Parámetro              | Valor         | Razón                                                                      |
|------------------------|---------------|----------------------------------------------------------------------------|
| Acelerómetro           | $\pm 4$ g     | Buen compromiso entre margen y resolución para las actividades propuestas. |
| Giroscopio             | $\pm 500$ °/s | Adecuado para movimientos corporales moderados.                            |
| Frecuencia de muestreo | 50 Hz         | Suficiente para capturar la dinámica de las actividades del taller.        |
| Bus I2C                | 400 kHz       | Permite leer seis canales con margen.                                      |
| Captura nominal P1C    | 20 s          | Equivale a 1000 muestras a 50 Hz.                                          |

$$N = f_s \times T = 50 \times 20 = 1000 \text{ muestras}$$

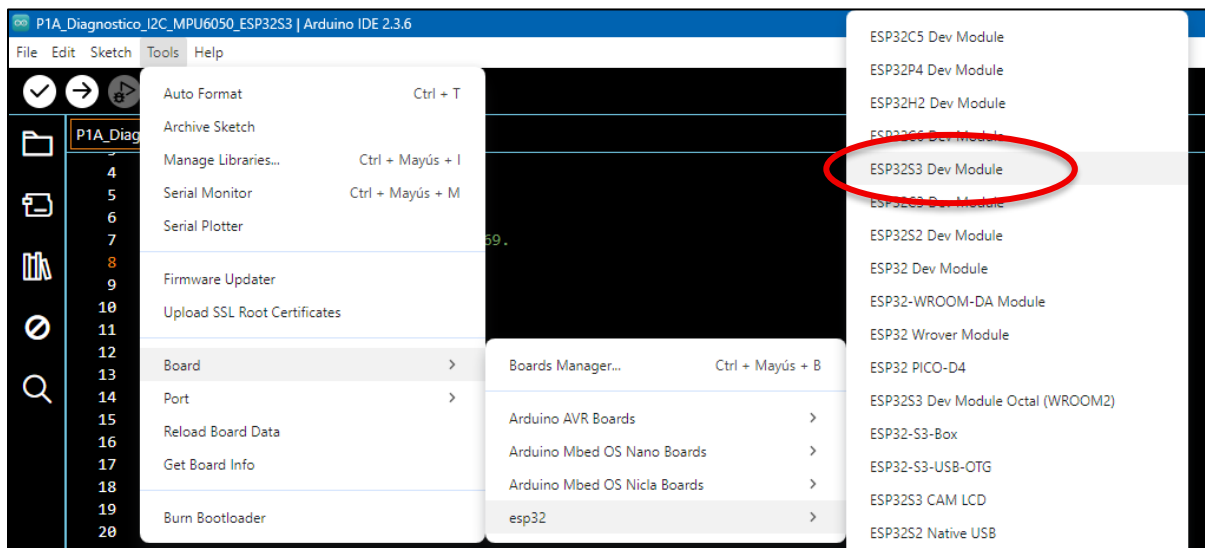
## Experimentos

### A. Diagnóstico del sensor

Antes de visualizar señales, se valida únicamente la capa de sensado. Esta separación permite distinguir rápidamente problemas de cableado/I2C, de comunicación o de software.

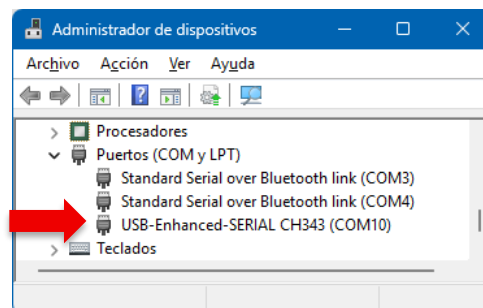
#### Procedimiento

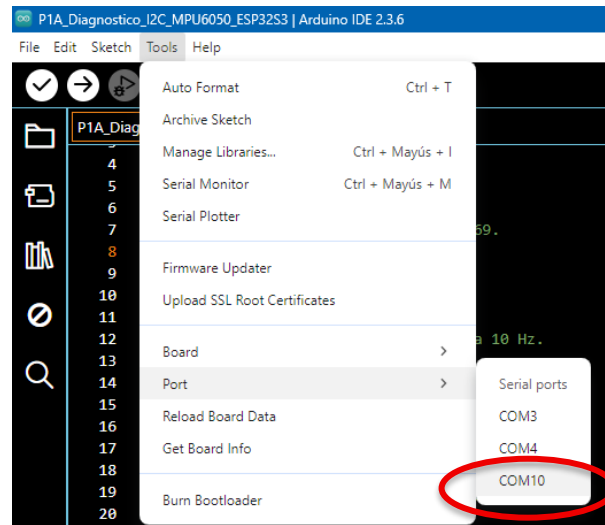
- Conecte el MPU6050 al ESP32-S3 mediante el cableado indicado en la tabla anterior.
- Conecte el ESP32-S3 a la computadora con el cable serial.
- Abra el IDE de Arduino.
- Abra el programa `P1A_Diagnostico_I2C_MPU6050_ESP32S3.ino`
- Confirme en el código que `SDA_PIN = 8` y `SCL_PIN = 9`.
- Vaya a **Tools** >> **Board**, seleccione **esp32** >> **ESP32S3 Dev Module**.



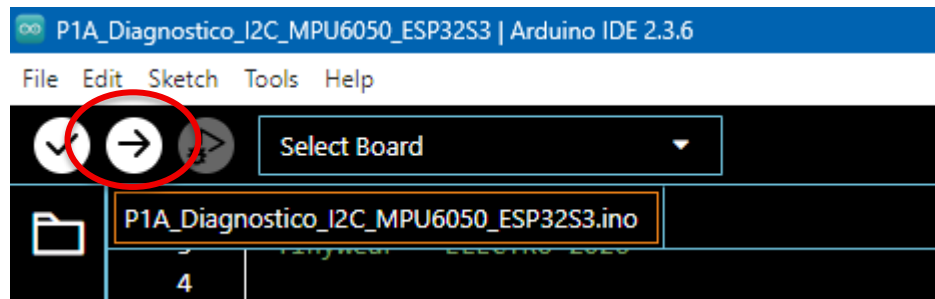
- En **Tools** >> **Board**, seleccione el puerto COM asignado a su tarjeta; en mi caso, es **COM10**.

**Nota:** si no recuerda el COM asignado a su tarjeta, lo puede consultar en el **Administrador de dispositivos**:



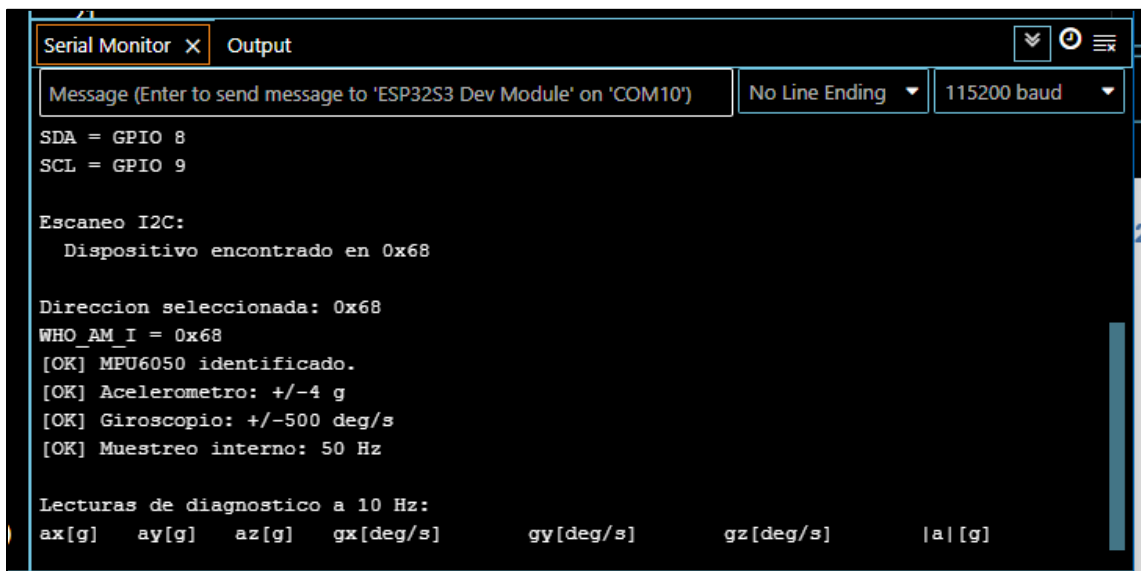


h. Compile y cargue el programa en el ESP32-S3.



i. En **Tools >> Serial Monitor**, seleccione el BaudRate en **115200 bit/s**.

## Salida esperada



- j. Compruebe que WHO\_AM\_I reporte 0x68 y que el firmware muestre “MPU6050 OK”.
- k. Al final del código, descomente la función **Serial.printf()**; para ello, borre los comentarios */\* y \*/*.

```

227  /*Serial.printf(
228      "%.3f\t%.3f\t%.3f\t%.2f\t%.2f\t%.2f\t%.3f\n",
229      ax, ay, az,
230      gx, gy, gz,
231      amag
232  );*/
233  }
234

```

```

227  Serial.printf(
228      "%.3f\t%.3f\t%.3f\t%.2f\t%.2f\t%.2f\t%.3f\n",
229      ax, ay, az,
230      gx, gy, gz,
231      amag
232  );
233  }
234

```

- l. Compile y cargue el programa en el ESP32-S3.
- m. Deje el sensor inmóvil y verifique que **|a|** sea aproximadamente **1 g**.

$$|a| = \sqrt{a_x^2 + a_y^2 + a_z^2} \approx 1 \text{ g (sensor inmóvil)}$$

- n. Inclíne y gire el módulo para comprobar que los seis canales respondan.

## Salida esperada

| Serial Monitor X Output                                            |       |       |      |       |       |       | ⌵ ⌚ ☰            |               |
|--------------------------------------------------------------------|-------|-------|------|-------|-------|-------|------------------|---------------|
| Message (Enter to send message to 'ESP32S3 Dev Module' on 'COM10') |       |       |      |       |       |       | No Line Ending ▾ | 115200 baud ▾ |
| -0.185                                                             | 0.091 | 0.813 | 0.06 | -0.73 | -0.93 | 0.838 |                  |               |
| -0.188                                                             | 0.089 | 0.810 | 0.06 | -0.67 | -1.02 | 0.837 |                  |               |
| -0.187                                                             | 0.089 | 0.815 | 0.08 | -0.73 | -1.01 | 0.840 |                  |               |
| -0.185                                                             | 0.088 | 0.818 | 0.09 | -0.70 | -0.96 | 0.843 |                  |               |
| -0.185                                                             | 0.092 | 0.815 | 0.05 | -0.70 | -0.95 | 0.841 |                  |               |
| -0.188                                                             | 0.090 | 0.817 | 0.09 | -0.67 | -1.02 | 0.843 |                  |               |
| -0.185                                                             | 0.092 | 0.813 | 0.09 | -0.76 | -1.01 | 0.839 |                  |               |
| -0.185                                                             | 0.090 | 0.812 | 0.06 | -0.67 | -0.99 | 0.838 |                  |               |
| -0.186                                                             | 0.087 | 0.814 | 0.08 | -0.70 | -0.98 | 0.840 |                  |               |
| -0.185                                                             | 0.088 | 0.815 | 0.06 | -0.72 | -0.99 | 0.840 |                  |               |

## B. Visualización de las seis señales

Una vez validado el sensor, se adquieren **ax**, **ay**, **az**, **gx**, **gy** y **gz** a 50 Hz y se envían por USB al Arduino Serial Plotter. En esta etapa, todavía no se utiliza Wi-Fi.

### Procedimiento

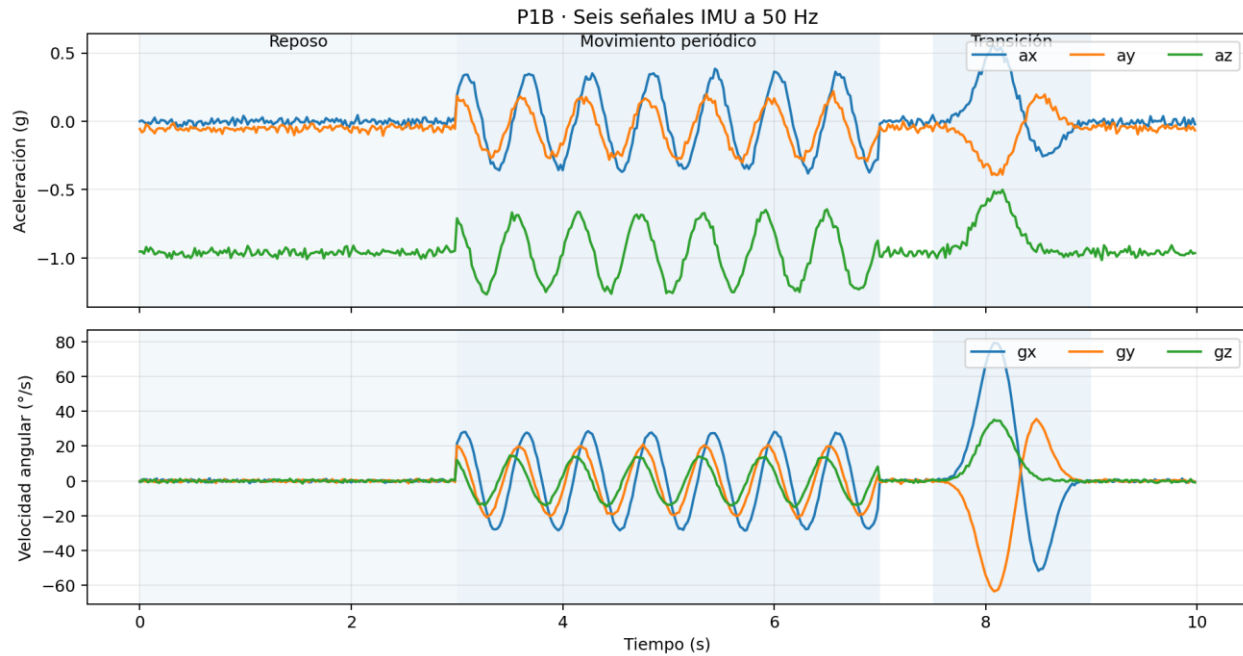
- Abra el programa **P1B\_SerialPlotter\_6Canales\_MPU6050\_ESP32S3.ino**.
- En **Tools >> Board**, seleccione **esp32 >> ESP32S3 Dev Module**.
- En **Tools >> Port**, seleccione el puerto COM asignado a su tarjeta; en mi caso, es **COM10**.
- Compile y cargue el programa en el ESP32-S3.
- Vaya a **Tools > Serial Plotter** y seleccione 115200 bit/s.
- Mantenga el sensor inmóvil durante 5–10 s y observe las seis curvas.
- Incline lentamente el módulo y observe las respuestas de **ax**, **ay** y **az**.
- Gire el módulo alrededor de diferentes ejes y observe las respuestas de **gx**, **gy** y **gz**.
- Coloque TinyWear en la cintura y realice las siguientes actividades: en reposo/de pie, caminar y sentarse/levantarse.
- Identifique y apunte qué señales parecen más informativas para cada actividad.

### Salida esperada



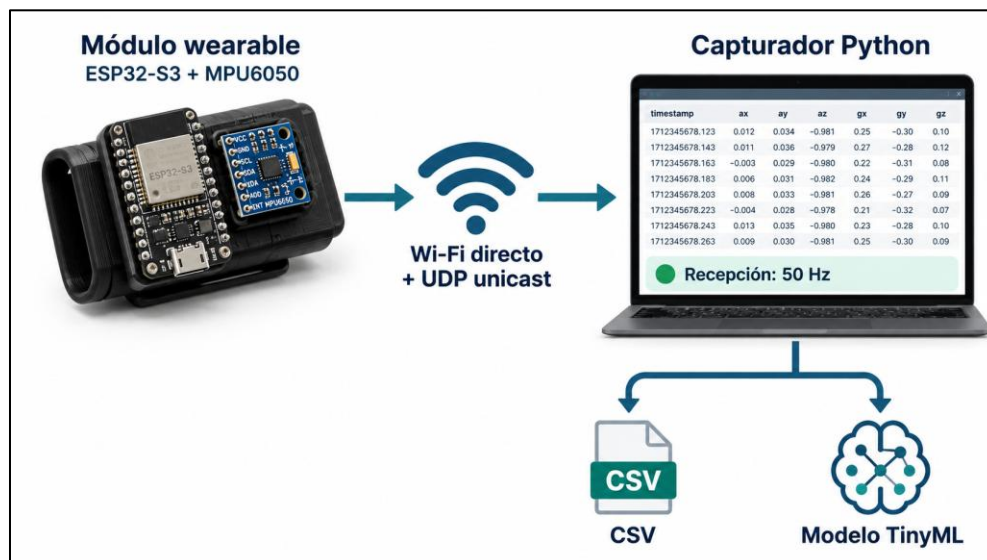
**Nota sobre las escalas.** El acelerómetro se expresa en g y el giroscopio en  $^{\circ}/s$ . Arduino Serial Plotter muestra todas las series sobre el mismo eje vertical; durante movimientos con velocidad angular elevada, las curvas del acelerómetro pueden verse visualmente comprimidas. Esto no implica pérdida de información.

**Ejemplo didáctico** de seis canales IMU muestreados a 50 Hz. Las amplitudes reales dependen del participante, la orientación y el montaje.



### C. TinyWear inalámbrico: Wi-Fi + UDP unicast

Después de comprobar el sensor y comprender las seis señales, se incorpora el enlace inalámbrico. El ESP32-S3 crea su propia red Wi-Fi y envía cada paquete directamente a la laptop registrada en Python.

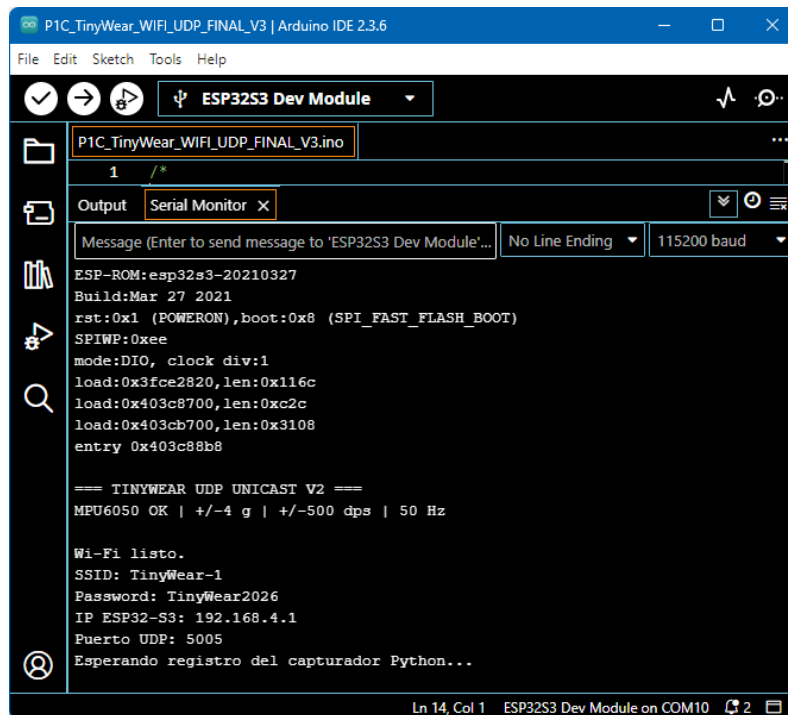


## Configuración de red

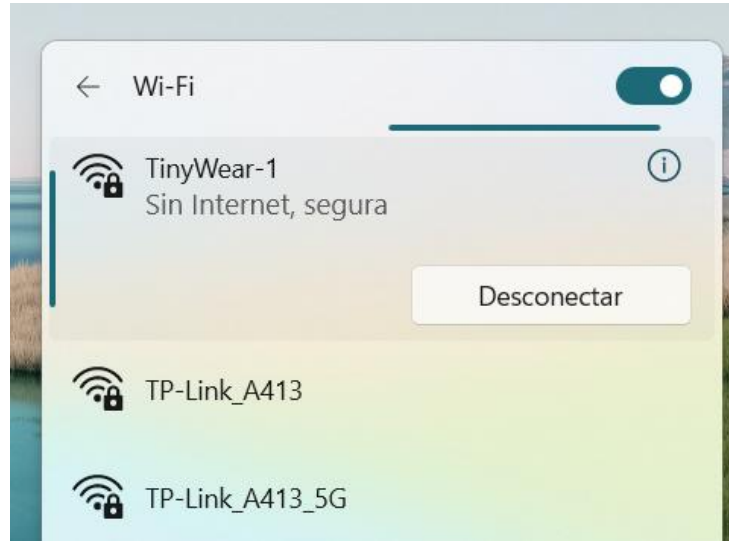
| Elemento        | Valor        |
|-----------------|--------------|
| SSID            | TinyWear-1   |
| Contraseña      | TinyWear2026 |
| IP del ESP32-S3 | 192.168.4.1  |
| Puerto UDP      | 5005         |
| Frecuencia      | 50 Hz        |

## Procedimiento

- Abra el programa **P1C\_TinyWear\_WIFI\_UDP\_FINAL\_V3.ino**.
- En el menú **Tools >> Board**, seleccione **esp32 >> ESP32S3 Dev Module**.
- En **Tools >> Port**, seleccione el puerto COM asignado a su tarjeta; en mi caso, es **COM10**.
- Compile y cargue el programa en el ESP32-S3.
- Vaya a **Tools > Serial Monitor** y seleccione 115200 bit/s.
- Pulse el botón RESET del ESP32-S3.
- Verifique “**MPU6050 OK**” y la creación de la red **TinyWear-1**.



- h. Conecte su PC a la red **TinyWear-1**; la contraseña es **TinyWear2026**. Aparecerá “Sin Internet”.



- i. Ejecute el programa **Prueba\_UDP\_Unicast\_FINAL.py**.  
j. Compruebe una tasa de 50 paquetes/s y pérdidas cercanas a cero.

## Salida esperada

```

C:\Windows\py.exe
250 | seq= 249 | 13.8 pkt/s | perdidos=0
TinyWear ACK recibido ✓
300 | seq= 299 | 15.7 pkt/s | perdidos=0
350 | seq= 349 | 17.4 pkt/s | perdidos=0
TinyWear ACK recibido ✓
400 | seq= 399 | 18.9 pkt/s | perdidos=0
450 | seq= 449 | 20.3 pkt/s | perdidos=0
TinyWear ACK recibido ✓
500 | seq= 499 | 21.6 pkt/s | perdidos=0
550 | seq= 549 | 22.8 pkt/s | perdidos=0
TinyWear ACK recibido ✓
600 | seq= 599 | 23.9 pkt/s | perdidos=0
650 | seq= 649 | 24.9 pkt/s | perdidos=0
TinyWear ACK recibido ✓
700 | seq= 699 | 25.8 pkt/s | perdidos=0
750 | seq= 749 | 26.7 pkt/s | perdidos=0
TinyWear ACK recibido ✓
800 | seq= 799 | 27.5 pkt/s | perdidos=0
850 | seq= 849 | 28.2 pkt/s | perdidos=0
TinyWear ACK recibido ✓
900 | seq= 899 | 28.9 pkt/s | perdidos=0
950 | seq= 949 | 29.6 pkt/s | perdidos=0
TinyWear ACK recibido ✓
1000 | seq= 999 | 30.2 pkt/s | perdidos=0
1050 | seq= 1049 | 30.8 pkt/s | perdidos=0
TinyWear ACK recibido ✓
1100 | seq= 1099 | 31.3 pkt/s | perdidos=0
1150 | seq= 1149 | 31.8 pkt/s | perdidos=0
TinyWear ACK recibido ✓

```

- k. Cierre la prueba de consola antes de abrir el capturador gráfico.

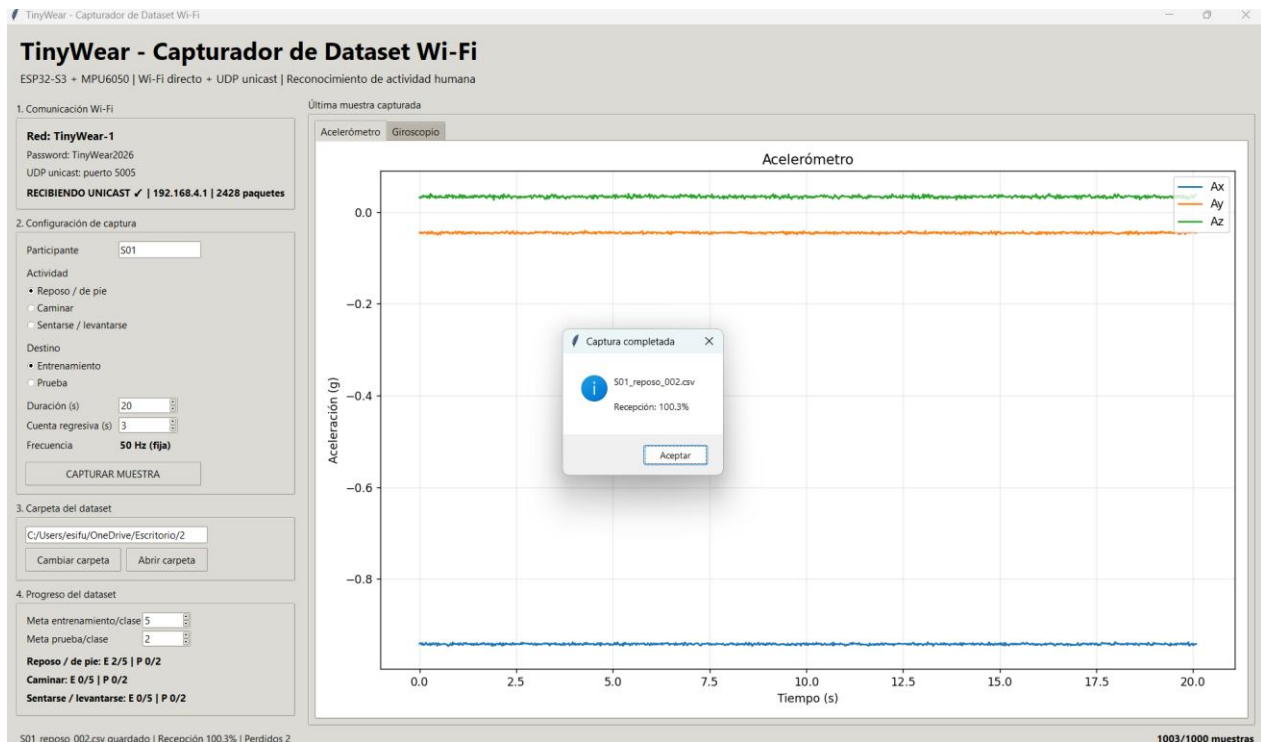
## D. Captura y visualización de una muestra

El programa **Capturador** recibe las lecturas, muestra el estado de la comunicación y guarda exactamente  $N = \text{duración} \times 50$  muestras por captura.

### Procedimiento

- Ejecute el programa **Capturador\_Dataset\_TinyWear\_WIFI\_FINAL\_V3.py**  
Compruebe que aparezca “RECIBIENDO UNICAST”.
- Seleccione participante (por ejemplo S01).
- Cree una carpeta donde guardará los archivos.
- Seleccione una actividad: reposo/de pie, caminar o sentarse/levantarse.
- Seleccione entrenamiento o prueba.
- Use 20 s como duración nominal y mantenga la actividad durante toda la captura.
- Compruebe que el archivo contenga exactamente 1000 muestras.
- Acepte la captura si la tasa de recepción es  $\geq 98\%$ ; de lo contrario, repita.

### Salida esperada

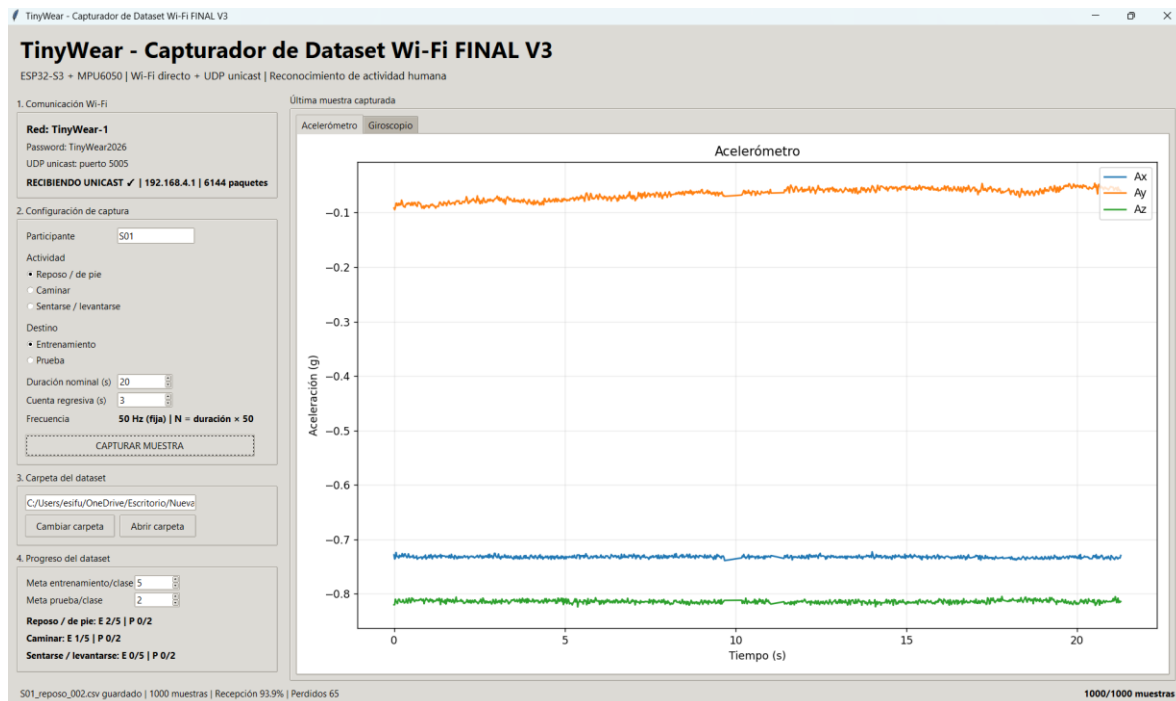


## Protocolo de prueba

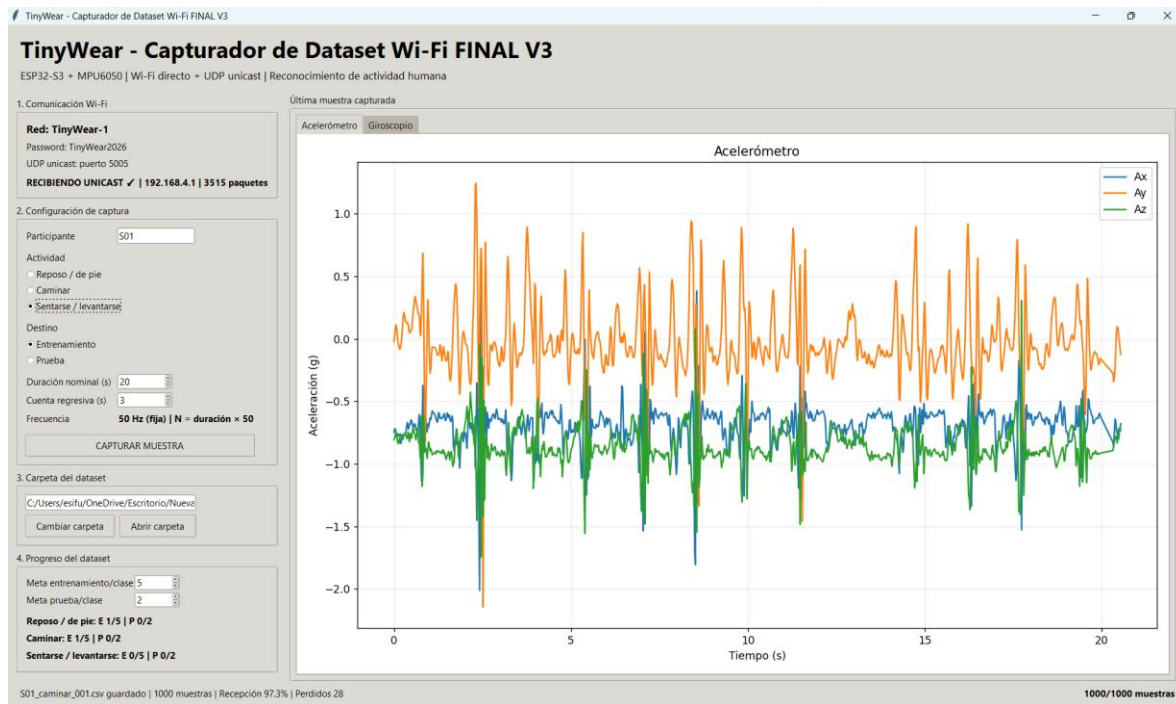
1. Trabaje en parejas: una persona utiliza TinyWear y la otra opera la computadora.
2. Coloque TinyWear en el lado derecho de la cintura con la misma orientación para todas las pruebas.
3. Seleccione el participante (por ejemplo S01).
4. Seleccione la actividad: reposo/de pie, caminar o sentarse/levantarse.
5. Elija entrenamiento o prueba.
6. Realice la cuenta regresiva y mantenga la actividad durante la captura.
7. Compruebe que se guarden exactamente 1000 muestras para una captura de 20 s.
8. Repita la captura si la recepción es menor a 98 % o si el movimiento no corresponde a la etiqueta seleccionada.

## Salida esperada

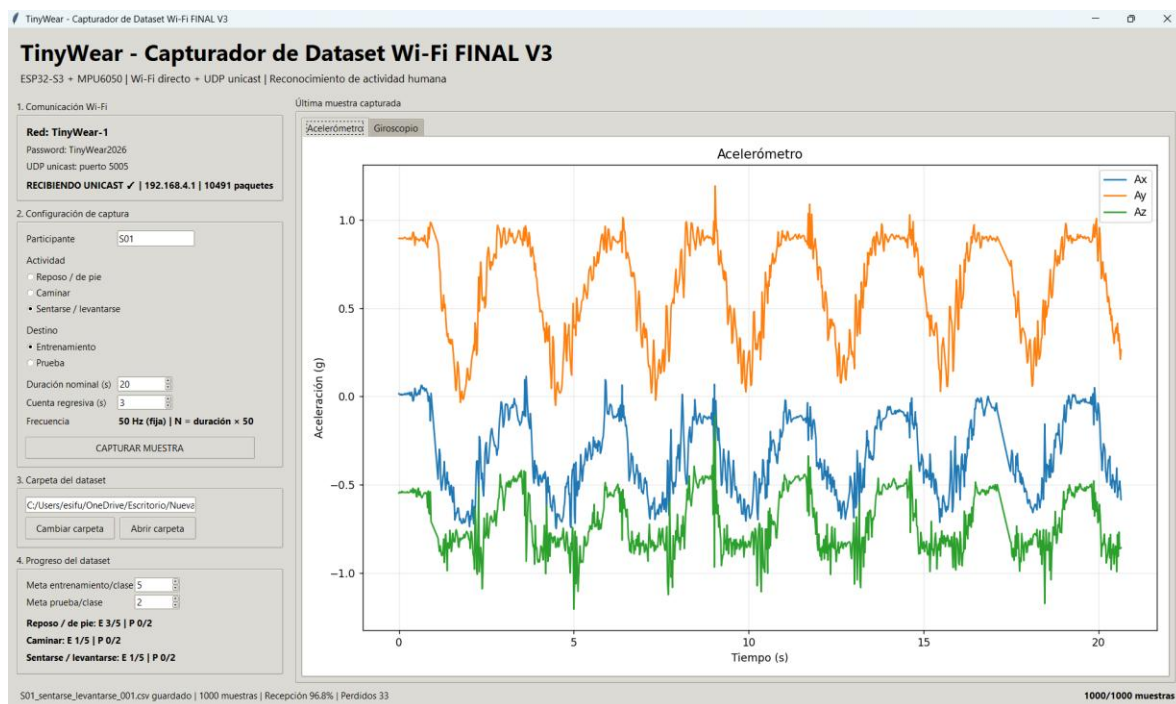
### Reposo



## Caminar



## Sentarse/ de pie



## Cuestionario de evaluación

1. ¿Qué ventaja hay en comprobar primero el bus I2C antes de utilizar Wi-Fi?
2. ¿Por qué  $|a|$  es aproximadamente 1 g cuando el sensor está inmóvil?
3. ¿Qué diferencia física existe entre una señal de aceleración y una señal de velocidad angular?
4. ¿Qué señales cambian más al pasar de reposo a caminar: acelerómetro, giroscopio o ambas?
5. ¿La transición de sentarse/levantarse puede reconocerse con una muestra instantánea o requiere observar un intervalo de tiempo?
6. ¿Qué efecto tendría cambiar TinyWear del lado derecho al izquierdo de la cintura?
7. ¿Qué función cumple UDP unicast en P1C?
8. ¿Qué efecto tendría cambiar la orientación de TinyWear entre una captura y otra?

## Conclusiones

P1 recorre el sistema por capas, comenzando por la validación del hardware y el bus I2C. Luego, se verifican las seis señales de la IMU a 50 Hz, y finalmente, se incorpora el enlace Wi-Fi/UDP junto con la captura en Python. La comunicación Wi-Fi directa facilitó enviar las muestras a una laptop y crear archivos CSV etiquetados. Así, se establece una base sólida y confiable para construir el conjunto de datos de P2.

## Evidencia del participante

- Captura del Monitor Serial de P1A mostrando la dirección I2C y WHO\_AM\_I correctos.
- Captura del Serial Plotter de P1B con las seis señales visibles.
- Salida de Prueba\_UDP\_Unicast\_FINAL.py con una tasa cercana a 50 paquetes/s.
- Captura del programa en Python que recibe datos.
- Al menos un archivo CSV por actividad.
- Respuestas al cuestionario de evaluación.

## Anexo A. Archivos utilizados

| Archivo                                        | Uso                                                                               |
|------------------------------------------------|-----------------------------------------------------------------------------------|
| P1A_Diagnostico_I2C_MPU6050_ESP32S3.ino        | Escaneo I2C, WHO_AM_I, configuración y lecturas de diagnóstico a 10 Hz.           |
| P1B_SerialPlotter_6Canales_MPU6050_ESP32S3.ino | Adquisición a 50 Hz y visualización de ax, ay, az, gx, gy y gz en Serial Plotter. |
| P1C_TinyWear_WIFI_UDP_FINAL_V3.ino             | Firmware validado para adquisición IMU y transmisión por Wi-Fi/UDP unicast.       |
| Prueba_UDP_Unicast_FINAL.py                    | Verificación del enlace, tasa de paquetes y pérdidas.                             |
| Capturador_Dataset_TinyWear_WIFI_FINAL_V3.py   | Captura etiquetada, control de calidad y generación de CSV.                       |